Data Science with Python

Dr. Abdul Wahid
Department of Computer Science and Engineering
IIIT Dharwad, Karnataka

E-mail: abdul.wahid@iiitdwd.ac.in

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY

Data Loading, Storage, and File Formats

Accessing data is first step to any data analysis step!

Reading Data- Text Format

Parsing functions in pandas

Function	Description
`read_csv`	Load delimited data from a file, URL, or file-like object; use comma as default delimiter
`read_table`	Load delimited data from a file, URL, or file-like object; use tab (`'\t'`) as default delimiter
`read_fwf`	Read data in fixed-width column format (i.e., no delimiters)
`read_clipboard`	Version of `read_table` that reads data from the clipboard; useful for converting tables from web pages
`read_excel`	Read tabular data from an Excel XLS or XLSX file
`read_hdf`	Read HDF5 files written by pandas
`read_html`	Read all tables found in the given HTML document
`read_json`	Read data from a JSON (JavaScript Object Notation) string representation
`read_msgpack`	Read pandas data encoded using the MessagePack binary format
`read_pickle`	Read an arbitrary object stored in Python pickle format

Function	Description
`read_sas`	Read a SAS dataset stored in one of the SAS system's custom storage formats
`read_sql`	Read the results of a SQL query (using SQLAlchemy) as a pandas DataFrame
`read_stata`	Read a dataset from Stata file format
`read_feather`	Read the Feather binary file format

Reading and Writing Data in Text Format

- Pandas has number of functions for reading tabular data as DataFrame object. 'read_csv' and 'read_table' are most likely to be used.
- All the functions in general convert text data into a DataFrame. The optional arguments in these functions can fall into a few categories:
  - Indexing
    - Can treat one or more columns as the returned DataFrame, and whether to get column names from the file, the user, or not at all.
  - Type inference and data conversion
    - This includes the user-defined value conversions and custom list of missing value markers.

Reading and Writing Data in Text Format

- Datetime parsing
  - Includes combining capability, including combining date and time information spread over multiple columns into a single column in the result.
- Iterating
  - Support for iterating over chunks of very large files.
- Unclean data issues
  - Skipping rows or a footer, comments, or other minor things like numeric data with thousands separated by commas.

Reading and Writing Data in Text Format

```
!cat /Users/abdulwahid/ex1.csv  # For macOS/Linux
#!type "C:\Users\abdulwahid\ex1.csv"  # For Windows (cmd)
```

```
a,b,c,d,message
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```

- If the file is comma-delimited, we can just use read_csv to read it.
- We can also use read_table with specific delimiter (here ,).
  - read_table: working with non-CSV tabular files (e.g., tab-separated values).
  - We need to explicitly define a different delimiter (e.g., ;, |, etc.).

Reading and Writing Data in Text Format

```
import pandas as pd
df = pd.read_csv('ex1.csv')  # Load the file
df.head()  # Preview first few rows
```

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

```
pd.read_table('ex1.csv',sep=',')
```

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

Reading and Writing Data in Text Format

- A file will not always have a header row.

```
!cat /Users/abdulwahid/ex2.csv  # For macOS/Linux
```

```
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```

- For this we can either allow pandas to assign default names or specify the names ourselves.

Reading and Writing Data in Text Format

- A file will not always have a header row.

```
!cat /Users/abdulwahid/ex2.csv  # For macOS/Linux
```

```
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```

- For this we can either allow pandas to assign default names or specify the names ourselves.
- Normally, pd.read_csv() assumes the **first row contains column names**.

Reading and Writing Data in Text Format

- Normally, pd.read_csv() assumes the first row contains column names.

```
pd.read_csv('ex2.csv')
```

	1	2	3	4	hello
0	5	6	7	8	world
1	9	10	11	12	foo

```
pd.read_csv('ex2.csv', header=None)
```

	0	1	2	3	4
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

- With header=None, pandas treats all rows as data and assigns default numerical column names (0, 1, 2, ...).

Reading and Writing Data in Text Format

- A file will not always have a header row; we are assigning the names.

```
pd.read_csv('ex2.csv', names=['a','b','c','d','message'])
```

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

- We want a specific column to be taken as index of the DataFrame, we can specify the Column name in the 'index_col' parameter. **Or specify the index number of the column**.

```
names = ['a','b','c','d','message']
pd.read_csv('ex2.csv', names=names, index_col='message')
#pd.read_csv('ex2.csv', names=names, index_col=4)
```

	a	b	c	d
message				
hello	1	2	3	4
world	5	6	7	8
foo	9	10	11	12

Reading and Writing Data in Text Format

- To form hierarchical index from multiple columns, pass a list of column numbers or names in that order.

```
!type examples\csv_mindex.csv
```

```
key1,key2,value1,value2
one,a,1,2
one,b,3,4
one,c,5,6
one,d,7,8
two,a,9,10
two,b,11,12
two,c,13,14
two,d,15,16
```

```
parsed = pd.read_csv('examples/csv_mindex.csv', index_col=['key1', 'key2'])
parsed
```

		value1	value2
key1	**key2**		
one	**a**	1	2
	b	3	4
	c	5	6
	d	7	8
two	**a**	9	10
	b	11	12
	c	13	14
	d	15	16

Reading and Writing Data in Text Format

- When dealing with inconsistent delimiters in a file, we can use regular expressions (**regex**) to handle cases where columns are separated by variable amounts of whitespace.

- If a file has **inconsistent spaces** between columns, we can use \s+ (which matches one or more spaces) as the delimiter.

```
list(open('ex3.txt'))
```

```
['            A         B         C\n',
 'aaa -0.264438 -1.026059 -0.619500\n',
 'bbb  0.927272  0.302904 -0.032399\n',
 'ccc -0.264273 -0.386314 -0.217601\n',
 'ddd -0.871858 -0.348382  1.100491\n']
```

```
#result = pd.read_table('ex3.txt', sep='\s+') # Python treats \s as an escape sequence
result = pd.read_table('ex3.txt', sep=r'\s+')
result
```

	A	B	C
aaa	-0.264438	-1.026059	-0.619500
bbb	0.927272	0.302904	-0.032399
ccc	-0.264273	-0.386314	-0.217601
ddd	-0.871858	-0.348382	1.100491

- Why aaa, bbb, ccc, and ddd are automatically set as the index?

Reading and Writing Data in Text Format

- `pd.read_table()` infers the structure of the file.
- The first row (A, B, C) is treated as column headers.
- The first column (aaa, bbb, ccc, ddd) does not have a header, **so pandas assumes it to be the index.**

Reading and Writing Data in Text Format

- Parser function can have multiple arguments to handle wide variety of exception file formats.
- We can skip rows in a file with 'skiprows'.

```
!type examples\ex4.csv
```

```
# hey!
a,b,c,d,message
# just wanted to make things more difficult for you
# who reads CSV files with computers, anyway?
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```

```
pd.read_csv('examples/ex4.csv', skiprows=[0,2,3])
```

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

Example

```
df_excel = pd.read_excel('students.xlsx')
print("\nExcel Data:")
print(df_excel)
```

```
# You may need to install the Excel engine:
pip install openpyxl
```

```
[
  {"Name": "Alice", "Age": 22, "Marks": 85},
  {"Name": "Bob", "Age": 23, "Marks": 78},
  {"Name": "Charlie", "Age": 21, "Marks": 90}
]
```

```python
import json

# Reading JSON file as Python list
with open('students.json', 'r') as f:
    data_json = json.load(f)

print("\nJSON Data (raw):")
print(data_json)

# Convert to DataFrame
df_json = pd.DataFrame(data_json)
print("\nJSON Data (DataFrame):")
print(df_json)
```

```python
import pandas as pd

url = 'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv'
df = pd.read_csv(url)

print("Iris Data (first 5 rows):")
print(df.head())
```

```
Iris Data (first 5 rows):
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
```

Exercise 1

- **Dataset**: Titanic Dataset
  📥 **URL**: https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv
- **Tasks**:
  - Read the dataset using `pd.read_csv()`.
  - Display the first 10 rows.
  - Show the column names and data types.
  - Count how many passengers survived vs. not survived.

Exercise 2

- **Dataset**: Air Travel Data
  📥 **URL**: https://people.sc.fsu.edu/~jburkardt/data/csv/airtravel.csv
- **Tasks**:
  - Load the dataset using `pd.read_csv()`.
  - Display the shape and basic statistics.
  - Plot the air travel in July across different years.

Writing Data- Text Format

Writing Data to Text Format

- Data can be exported to delimited format. The 'to_csv' method can write data to a comma-seperated file.
- Other delimiters can also be used with the 'sep' argument. We can display result on console using the 'sys.stdout' argument.
- Missing values appear as empty string in the oputput. We can denote them with a different sentinel value with 'na_rep' argument.

```
import pandas as pd
data = pd.read_csv('ex5.csv')
data
```

	something	a	b	c	d	message
0	one	1	2	3.0	4	NaN
1	two	5	6	NaN	8	world
2	three	9	10	11.0	12	foo

```
data.to_csv('out5.csv')
```

```
!cat out5.csv
```

```
,something,a,b,c,d,message
0,one,1,2,3.0,4,
1,two,5,6,,8,world
2,three,9,10,11.0,12,foo
```

Writing Data to Text Format

```
import sys
data.to_csv(sys.stdout, sep='|')
```

```
|something|a|b|c|d|message
0|one|1|2|3.0|4|
1|two|5|6||8|world
2|three|9|10|11.0|12|foo
```

```
data.to_csv(sys.stdout, na_rep='NULL')
```

```
,something,a,b,c,d,message
0,one,1,2,3.0,4,NULL
1,two,5,6,NULL,8,world
2,three,9,10,11.0,12,foo
```

Writing Data to Text Format

- By default, both row and column headers are written. But they can be disabled with the 'index=False' and 'header=False' argument for rows and columns respectively.
- We can also write a subset of the columns in the order we want.

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago']
}
df = pd.DataFrame(data)
df.head()
```

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Charlie	35	Chicago

```
df.to_csv('output.csv', index=False, header=False, columns=['City', 'Name'])
```

```
!cat output.csv
```

```
New York,Alice
Los Angeles,Bob
Chicago,Charlie
```

Quiz

- Hide both the column and row header in "ex5.csv" dataset and show the result.

Exercise

- Read the file Feedback.csv into a DataFrame and display the first 15 rows.
- Display the column names, number of rows, and any missing values.
- Find how many total responses were collected.
- Save only the feedback with a rating of **4 or above** into a new CSV file.

Where:

"Excellent": 5,
"Very Good": 4,
"Good": 3,
"Average": 2,
"Poor": 1

- Calculate the average rating across all responses.

Writing Data to Text Format

- pd.date_range(start_date, periods=n) creates a sequence of dates.
- '1/1/2000' is the starting date.
- periods=7 means we generate 7 consecutive dates.

```
import numpy as np

dates = pd.date_range('1/1/2000', periods=7)
ts = pd.Series(np.arange(7), index=dates)
ts.to_csv('tseries.csv', header=False)
```

```
ts
```

```
2000-01-01    0
2000-01-02    1
2000-01-03    2
2000-01-04    3
2000-01-05    4
2000-01-06    5
2000-01-07    6
Freq: D, dtype: int64
```

```
!cat tseries.csv
```

```
2000-01-01,0
2000-01-02,1
2000-01-03,2
2000-01-04,3
2000-01-05,4
2000-01-06,5
2000-01-07,6
```

JSON Data

- JSON (JavaScript Object Notation) has become one of the standard formats for sending data via HTTP between browsers and applications.
- It is a much more free-form data format than CSV.
- It is very valid in Python except for a few exceptions:
  - Its 'null' value that is not recognized in Python.
  - Disallowing trailing commas at end of list
- The basic types in JSON are:

  1. Objects (dicts), 2. Arrays (lists), 3. Strings, 4. Numbers, 5. Booleans, 6. Nulls
  - All the keys in the object must be strings.

JSON Data

- There are several libraries for working with JSON files. One of them is 'json', which is built into standard library.
- We can use 'json.loads' to convert JSON string to Python DataFrame.
- 'json.dumps' converts Python object back to JSON.
- Use json.load() when you want raw dictionary data to process further in Python

```
obj = """
{"name": "Wes",
 "places_lived": ["United States", "Spain", "Germany"],
 "pet": null,
 "siblings": [{"name": "Scott", "age": 30, "pets": ["Zeus", "Zuko"]},
              {"name": "Katie", "age": 38,
               "pets": ["Sixes", "Stache", "Cisco"]}]
}
"""
```

JSON Data

```
import json

result = json.loads(obj)
result
```

```
{'name': 'Wes',
 'places_lived': ['United States', 'Spain', 'Germany'],
 'pet': None,
 'siblings': [{'name': 'Scott', 'age': 30, 'pets': ['Zeus', 'Zuko']},
  {'name': 'Katie', 'age': 38, 'pets': ['Sixes', 'Stache', 'Cisco']}]}
```

```
asjson = json.dumps(result)
asjson
```

```
'{"name": "Wes", "places_lived": ["United States", "Spain", "Germany"], "pet": null, "siblings": [{"name": "Scott", "age": 30, "pets": ["Zeu
s", "Zuko"]}, {"name": "Katie", "age": 38, "pets": ["Sixes", "Stache", "Cisco"]}]}'
```

```
siblings = pd.DataFrame(result['siblings'], columns=['name','age', 'pets'])
siblings
```

	name	age	pets
0	Scott	30	[Zeus, Zuko]
1	Katie	38	[Sixes, Stache, Cisco]

JSON Data

- 'pandas.read_json' can automatically convert JSON datasets into Series or DataFrame.
- The default options for this assume that each object in the JSON array is a row in the table.
- To export data from pandas to JSON, we can use 'to_json' method on a Series or DataFrame.
- Use pd.read_json() when you want a DataFrame for analysis and visualization.

XML and HTML : Web Scraping

```
import pandas as pd

tables = pd.read_html('examples/fdic_failed_bank_list.html')
len(tables)
```

```
1
```

```
failures = tables[0]
failures.head()
```

	Bank Name	City	ST	CERT	Acquiring Institution	Closing Date	Updated Date
0	Allied Bank	Mulberry	AR	91	Today's Bank	September 23, 2016	November 17, 2016
1	The Woodbury Banking Company	Woodbury	GA	11297	United Bank	August 19, 2016	November 17, 2016
2	First CornerStone Bank	King of Prussia	PA	35312	First-Citizens Bank & Trust Company	May 6, 2016	September 6, 2016
3	Trust Company Bank	Memphis	TN	9956	The Bank of Fayette County	April 29, 2016	September 6, 2016
4	North Milwaukee State Bank	Milwaukee	WI	20364	First-Citizens Bank & Trust Company	March 11, 2016	June 16, 2016

Exercises- Pandas

Data Cleaning- Handling Missing Data

Handling Missing Data

```
In [10]: string_data = pd.Series(['aardvark', 'artichoke', np.nan, 'avocado'])

In [11]: string_data
Out[11]:
0     aardvark
1    artichoke
2          NaN
3      avocado
dtype: object

In [12]: string_data.isnull()
Out[12]:
0    False
1    False
2     True
3    False
dtype: bool
```

```
In [13]: string_data[0] = None

In [14]: string_data.isnull()
Out[14]:
0     True
1    False
2     True
3    False
dtype: bool
```

NA handling methods

Argument	Description
`dropna`	Filter axis labels based on whether values for each label have missing data, with varying thresholds for how much missing data to tolerate.
`fillna`	Fill in missing data with some value or using an interpolation method such as `'ffill'` or `'bfill'`.
`isnull`	Return boolean values indicating which values are missing/NA.
`notnull`	Negation of `isnull`.

Filtering Out Missing Data

```
In [15]: from numpy import nan as NA

In [16]: data = pd.Series([1, NA, 3.5, NA, 7])

In [17]: data.dropna()
Out[17]:
0    1.0
2    3.5
4    7.0
dtype: float64
```

```
In [18]: data[data.notnull()]
Out[18]:
0    1.0
2    3.5
4    7.0
dtype: float64
```

Filling In Missing Data

- Rather than filtering out missing data (and potentially discarding other data along with it), you may want to fill in the “holes” in any number of ways. For most purposes, the fillna method is the workhorse function to use. Calling fillna with a constant replaces missing values with that value.

Argument	Description
`value`	Scalar value or dict-like object to use to fill missing values
`method`	Interpolation; by default `'ffill'` if function called with no other arguments
`axis`	Axis to fill on; default `axis=0`
`inplace`	Modify the calling object without producing a copy
`limit`	For forward and backward filling, maximum number of consecutive periods to fill

Data Transformation

- Transforming Data Using a Function or Mapping
- **Using map() with a Function**
  - The map() function applies a given function to each element of a **Pandas Series** (i.e., a single column).
  - This is useful when we want to **modify values conditionally**.

```
import pandas as pd
data = pd.Series([0, 10, 20, 30, 40])
def celsius_to_fahrenheit(c): # Function to convert Celsius to Fahrenheit
    return (c * 9/5) + 32
# Apply the function to each element
transformed_data = data.map(celsius_to_fahrenheit)
print(transformed_data)
```

```
0     32.0
1     50.0
2     68.0
3     86.0
4    104.0
dtype: float64
```

Data Transformation

- **Using map() with a Dictionary**
  - We can use a dictionary to map specific values to new values.
  - Any values not found in the dictionary remain NaN.

```
data = pd.Series(['NY', 'CA', 'TX', 'FL', 'WA'])
state_mapping = {
    'NY': 'New York',
    'CA': 'California',
    'TX': 'Texas'
}
# Apply the mapping
mapped_data = data.map(state_mapping)
print(mapped_data)
```

```
0      New York
1    California
2         Texas
3           NaN
4           NaN
dtype: object
```

Data Transformation

- We can use .fillna() to handle missing mappings:

```
mapped_data = data.map(state_mapping).fillna("Not defined")
print(mapped_data)
```

```
0       New York
1     California
2          Texas
3    Not defined
4    Not defined
dtype: object
```

Data Transformation

- **Handling Case Sensitivity**
  - If the data has inconsistent casing (e.g., "ny", "Ny", "NY"), mapping might fail.
  - We can convert everything to lowercase before mapping.

```
data = pd.Series(['ny', 'CA', 'Tx', 'FL', 'wa'])
state_mapping = {
    'ny': 'New York',
    'ca': 'California',
    'tx': 'Texas'
}
mapped_data = data.str.lower().map(state_mapping).fillna("Not defined")
print(mapped_data)
```

```
0        New York
1      California
2           Texas
3    Not defined
4    Not defined
dtype: object
```

Data Transformation

- **Using map() with a Lambda Function**
  - If we need custom transformations, we can use a lambda function inside map().

```
numbers = pd.Series([1, 2, 3, 4, 5])
labeled_data = numbers.map(lambda x: 'Even' if x % 2 == 0 else 'Odd')
print(labeled_data)
```

```
0     Odd
1    Even
2     Odd
3    Even
4     Odd
dtype: object
```

Quiz

Given a dataset containing student names, their ages, and attendance (%).

- **Classify each student based on their age** using the following categories:
  - **Child**: Age < 16
  - **Teen**: Age 16–19
  - **Adult**: Age 19–59
  - **Senior**: Age 60+
- **Categorize their attendance status** based on their attendance percentage:
  - **Good**: 75% and above
  - **Average**: 50% to 74%
  - **Poor**: Below 50%

Quiz- data

```
import pandas as pd

df = pd.DataFrame({
    'Student': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank'],
    'Age': [5, 16, 25, 45, 70, 19],
    'Attendance (%)': [95, 80, 60, 45, 85, 50]
})
```

Solution

```
import pandas as pd

# Creating a DataFrame with student data
df = pd.DataFrame({
    'Student': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank'],
    'Age': [5, 16, 25, 45, 70, 19],
    'Attendance (%)': [95, 80, 60, 45, 85, 50]
})
# Categorizing students based on age
df['Category'] = df['Age'].map(lambda x:
                               'Child' if x < 13 else
                               'Teen' if x < 20 else
                               'Adult' if x < 60 else
                               'Senior')
# Categorizing students based on attendance percentage
df['Attendance Status'] = df['Attendance (%)'].map(lambda x:
                                'Good' if x >= 75 else
                                'Average' if x >= 50 else
                                'Poor')
print(df)
```

Solution

	Student	Age	Attendance (%)	Category	Attendance Status
0	Alice	5	95	Child	Good
1	Bob	16	80	Teen	Good
2	Charlie	25	60	Adult	Average
3	David	45	45	Adult	Poor
4	Eve	70	85	Senior	Good
5	Frank	19	50	Teen	Average

Replacing Values in Pandas: fillna(), map(), and replace()

- The fillna method is a special case of more general values replacement.
- The map function modifies a subset of values but, 'replace' provides simpler and more flexible way to do so.

- We often need to replace missing or incorrect values in data.

  - fillna() is used for filling missing values (NaN).
  - map() can modify specific values in a single column (at a time).
  - replace() is more general and flexible, allowing us to replace multiple values easily.

Replacing Values in Pandas: replace()

```
df = pd.DataFrame({
    'Student': ['Alice', 'Bob', 'Charlie', 'David'],
    'Grade': ['A', 'B', 'C', 'A'],
    'Attendance': ['Good', 'Poor', 'Good', 'Average']
})
# Using replace() to change multiple values
df.replace({'A': 'Excellent', 'B': 'Good', 'C': 'Average', 'Poor': 'Needs Improvement'}, inplace=True)
print(df)
```

```
   Student      Grade         Attendance
0    Alice  Excellent               Good
1      Bob       Good  Needs Improvement
2  Charlie    Average               Good
3    David  Excellent            Average
```

Replacing Values in Pandas: replace() with NaN

```python
import pandas as pd
import numpy as np
# Creating a DataFrame with NaN values
df = pd.DataFrame({
    'Student': ['Alice', 'Bob', 'Charlie', 'David'],
    'Score': [85, np.nan, 90, np.nan],
    'Attendance': [np.nan, 'Good', 'Poor', 'Average']
})
# Replacing NaN values with default values
df.replace({np.nan: 'Missing'}, inplace=True)
print(df)
```

```
   Student    Score Attendance
0    Alice     85.0    Missing
1      Bob  Missing       Good
2  Charlie     90.0       Poor
3    David  Missing    Average
```

Replacing Values in Pandas

- To replace multiple values with a single value, pass a list followed by substitute value.
- To have different replacements for different values, pass list of substitutes.
- You can also pass a dict as argument to replace multiple substitutes.

```
data.replace([-999, -1000], np.nan)
```

```
0    1.0
1    NaN
2    2.0
3    NaN
4    3.0
dtype: float64
```

```
data.replace([-999, -1000], [np.nan, 0])
```

```
0    1.0
1    NaN
2    2.0
3    NaN
4    3.0
dtype: float64
```

```
data.replace({-999: np.nan, -1000: 0})
```

```
0    1.0
1    NaN
2    2.0
3    NaN
4    3.0
dtype: float64
```

Classroom Exercise: Online Retail Dataset

This dataset contains transactional data from a UK-based online retailer between 01/12/2010 and 09/12/2011. It includes information such as InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, and Country.

Access Dataset here

Classroom Exercise: Online Retail Dataset

1. Fill missing values in the `CustomerID` column using appropriate strategies.
2. Use `str.replace()` to clean and standardize text in the `Description` column.
3. Replace negative values in the `Quantity` column with `NaN` or appropriate values.

Data Visualization

Exploration vs. Presentation

Data Visualization is the process of taking a set of data and representing it in a visual format. Whenever you've made charts or graphs in past math or science classes, you've visualized data!

Visualization is used for two primary purposes: **exploration** and **presentation**.

Two types of visualization

Data exploration visualization: figuring out what is true

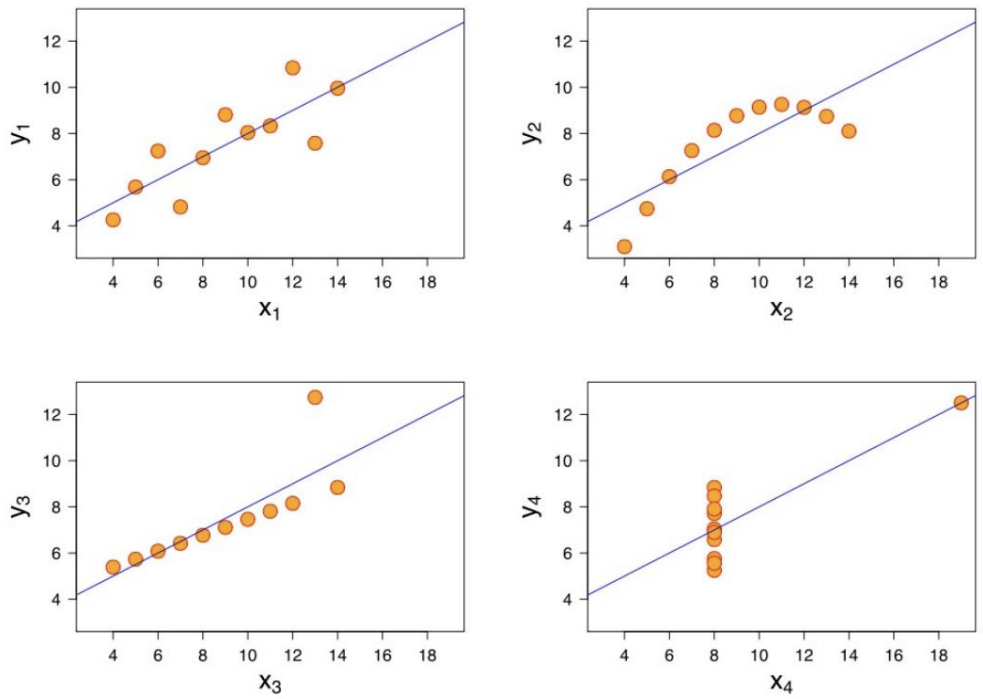
Data presentation visualization: convincing other people it is true

“Data exploration” is much broader than just visualization (most of the analysis techniques we will cover fit into it)

Data Exploration

In data exploration, charts created from data can provide information about that data beyond what is found in simple analyses alone.

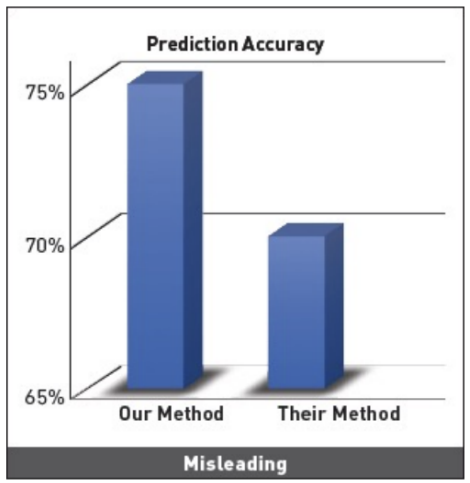
For example, the four graphs to the right all have the same mean and the same best-fit linear regression. But they tell very different stories.

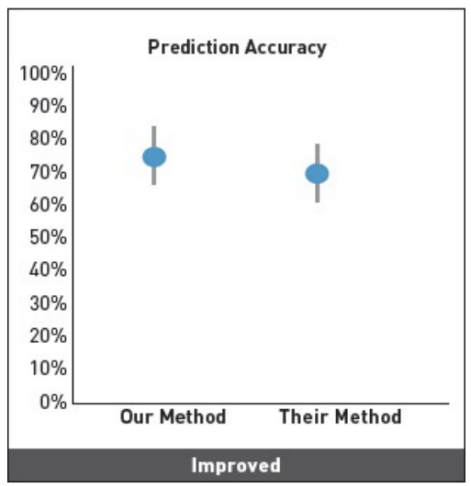


Data Presentation

In data presentation, you've already found an interesting pattern in the data and you need to make that pattern easily visible to other people.

In order to choose the best visualization for the job, consider the **type** of the data you're presenting (categorical, ordinal, or numerical), and how many **dimensions** of data you need to visualize.





https://interactions.acm.org/archive/view/july-august-2018/the-good-the-bad-and-the-biased

Importance of visualization

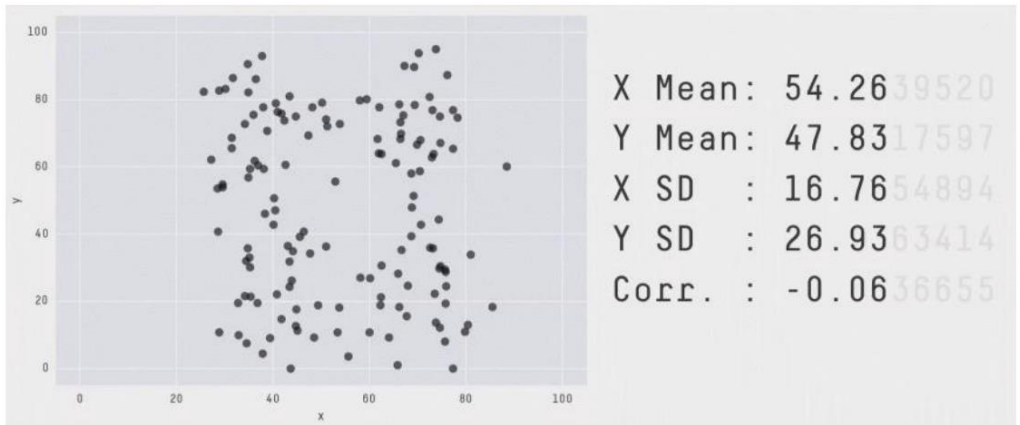
Before you run any analysis, build any machine learning system, etc, always visualize your data

If you can't identify a trend or make a prediction for your dataset, it's unlikely that an automated algorithm will

This is especially important to keep in mind as you hear stories of "superhuman" performance of AI methods (it is possible, but takes a long time, and is not the norm)

Visualization vs. statistics

Visualization almost always presents a more informative (though less quantitative) view of your data than statistics (the noun, not the field)



[Source: https://twitter.com/JustinMatejka/status/770682771656368128 Credit: @JustinMatejka, @albertocairo]

Data types

Nominal: categorical data, no ordering

Example – Pet: {dog, cat, rabbit, …}

Operations: $=, \neq$

Ordinal: categorical data, with ordering

Example – Rating: {1,2,3,4,5}

Operations: $=, \neq, \geq, \leq, >, <$

Interval: numerical data, zero doesn't mean zero "quantity"

Example – Temperature Fahrenheit

Operations: $=, \neq, \geq, \leq, >, <, +, -$

Ratio: numerical data, zero has meaning related to zero "quantity"

Example – Temperature Kelvin

Operations: $=, \neq, \geq, \leq, >, <, +, -, \div$

Visualization Types

Most discussion of visualization types emphasizes what elements the chart is trying to convey

Instead, we are going to focus on the type and dimensionality of the underlying data

Visualization types (not an exhaustive list):

- 1D: bar chart, pie chart, histogram
- 2D: scatter plot, line plot, box and whisker plot, heatmap
- 3D+: scatter matrix, bubble chart

One-Dimensional Data

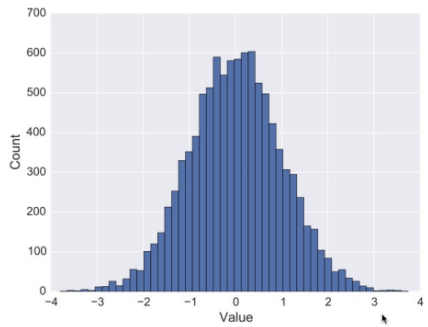
A **one-dimensional visualization** only visualizes a single feature of the dataset. For example:

"I want to know how many of each **product type** are in my data"

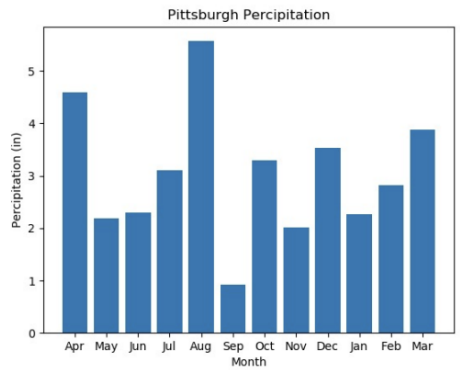
"I want to know the proportion of **people who have cats** in my data"

Charts for One-Dimensional Data

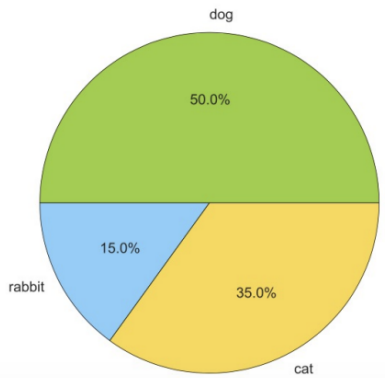
To visualize **numerical** data, use a **histogram**.



To visualize **ordinal** data, use a **bar chart**.



To visualize **categorical** data, use a **pie chart**.



Two-Dimensional Data

A **two-dimensional visualization** shows how two features in the dataset relate to each other. For example:

"I want to know the **cost** of each **product category** that we have"

"I want to know the **weight** of the animals that people own, by **pet species**"

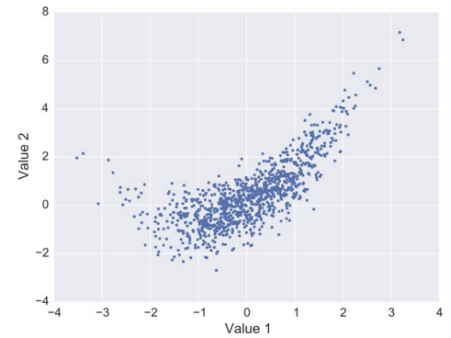
"I want to know how the **size** of the product affects **the cost of shipping**"

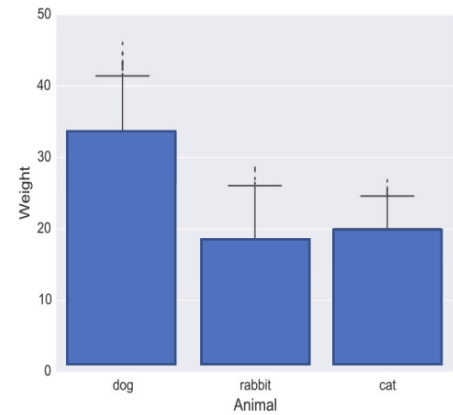
Charts for Two-Dimensional Data

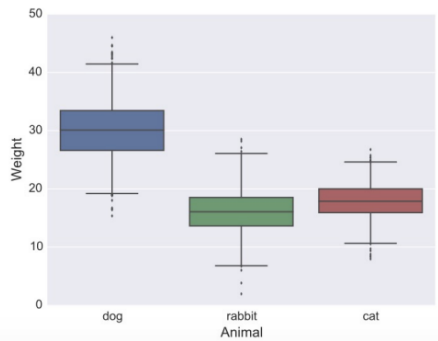
To analyze **numerical x numerical** data, use a **scatter plot**.

To analyze **numerical x ordinal/categorical** data, use a **bar chart** for averages or a **box-and-whiskers plot** for ranges.

It is difficult to analyze **ordinal/categorical x ordinal/categorical** data visually; use a table instead.







Three-Dimensional Data

A **three-dimensional** visualization tries to show the relationship between three different features at the same time. For example:

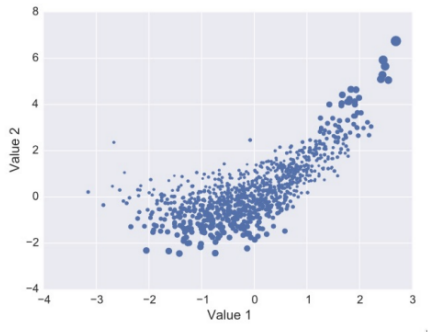
"I want to know the **cost** and the **development time** by **product category**"

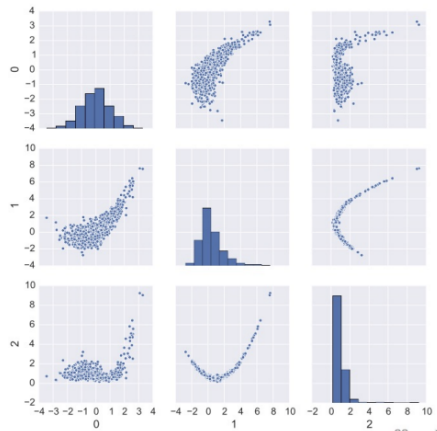
"I want to know the **weight** of the animals that people own and how much they **cost**, by **pet species**"

"I want to know how the **size** of the product and the **manufacturing location** affects **the cost of shipping**"

Charts for Three-Dimensional Data

To analyze **numerical x numerical x numerical** data, use a **bubble plot** to compare all three or a **scatter plot matrix** to compare all the pairs.





To analyze **numerical x numerical x ordinal/categorical** data, use a **colored scatter plot**.

Activity: Pick a Visualization

You do: for each of the problem prompts, determine the number of dimensions, then pick the **best** visualization to use based on the data types.

- graph the % of people who have gotten COVID vs. the % of people who have been vaccinated, separated by state
- graph the distribution of grades (72, 94, etc) in a class
- graph the ages of pets at a shelter compared to the species of pets

Coding Visualizations with Matplotlib

Matplotlib Makes Visualizations

The **matplotlib** library can be used to generate interesting visualizations in Python.

Matplotlib is **external** – you need to install it on your machine to run it. Use the `pip` command to do this.

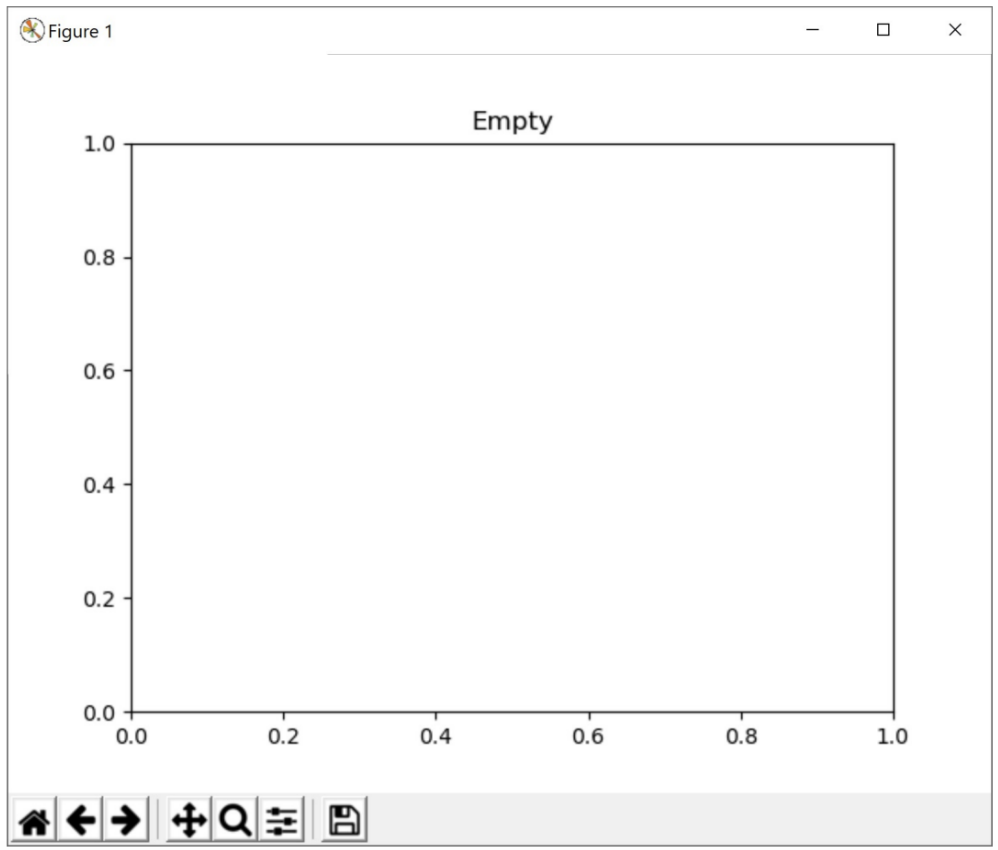
```
pip install matplotlib
```

Draw Visualizations on the Plot

Matplotlib visualizations can be broken down into several components. We'll mainly care about one: the **plot** (called `plt`). This is like Tkinter's canvas, except that we'll draw visualizations on it instead of shapes.

We can construct an (almost) empty plot with the following code. Note that matplotlib comes with built-in buttons that let you zoom, move data around, and save images.

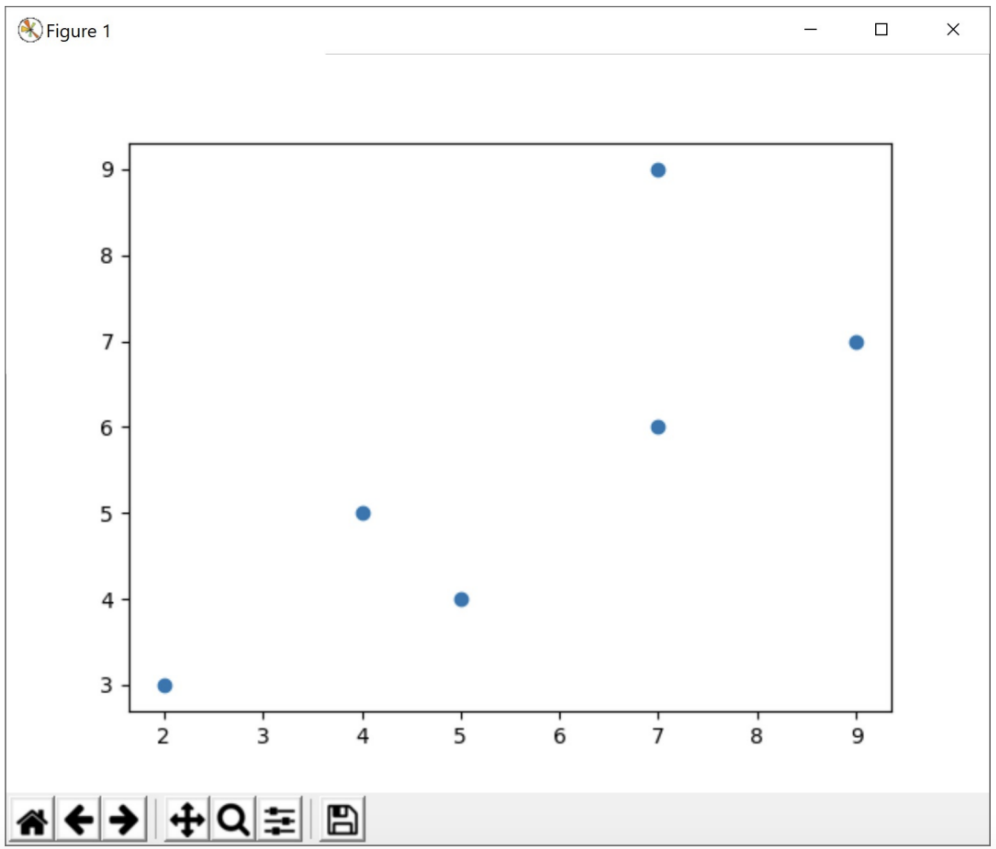
```
import matplotlib.pyplot as plt

plt.title("Empty")
plt.show()
```



Add Visualizations with Methods

There are lots of built-in methods that let you construct different types of visualizations. For example, to make a scatterplot use `plt.scatter(xValues, yValues)`.

```
x = [2, 4, 5, 7, 7, 9]
y = [3, 5, 4, 6, 9, 7]
plt.scatter(x, y)
plt.show()
```

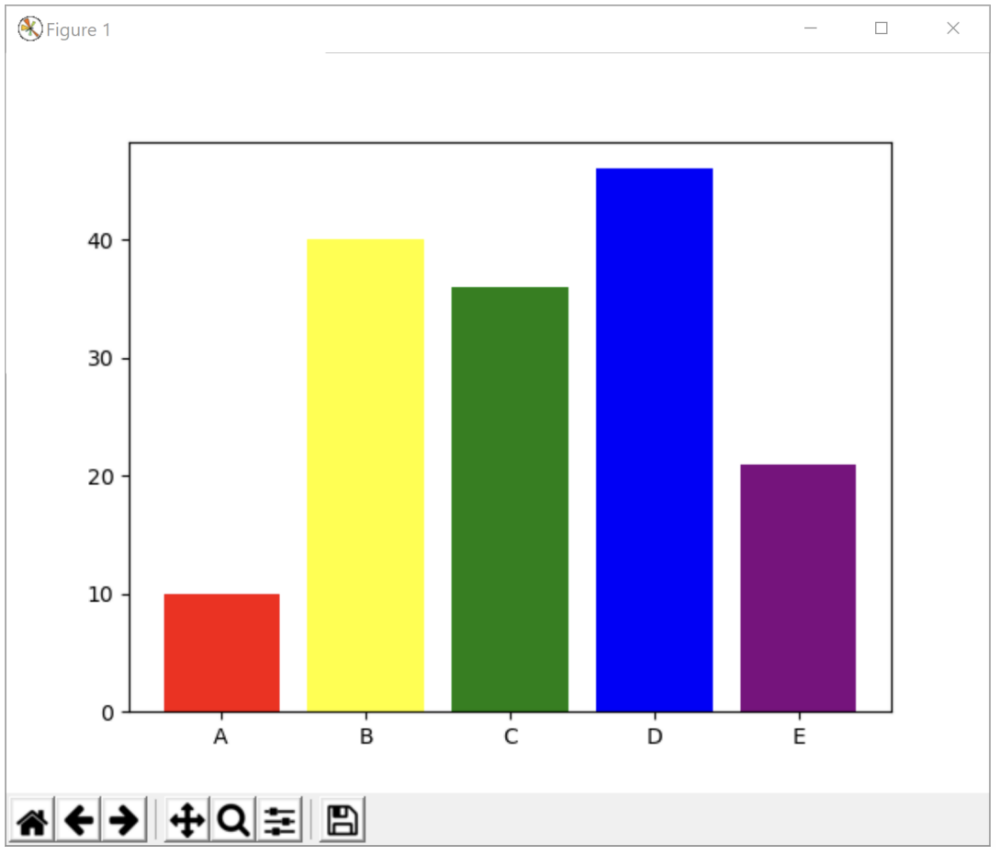


Visualization Methods have Keyword Args

You can **customize** how a visualization looks by adding **keyword arguments**. We used these in Tkinter to optionally change a shape's color or outline; in Matplotlib we can use them to add labels, error bars, and more.

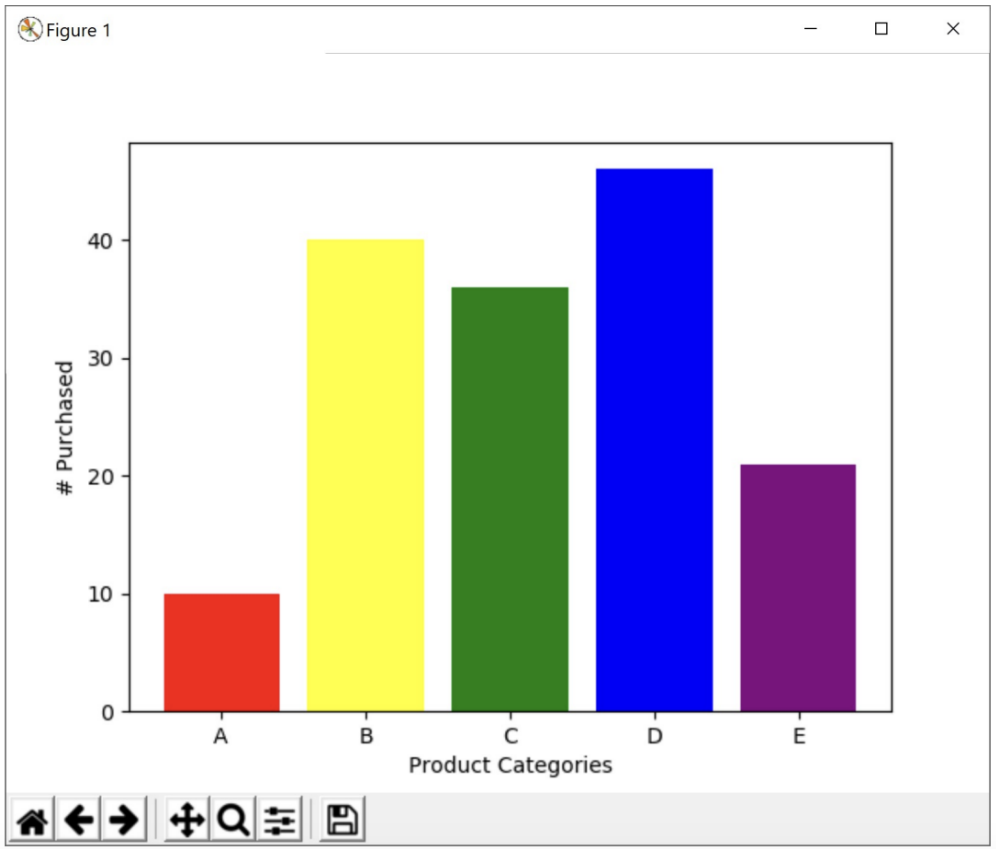
For example, we might want to create a bar chart (with `plt.bar`) with a unique color for each bar. Use the keyword argument `color` to set the colors.

```
labels = [ "A", "B", "C", "D", "E" ]
yValues = [ 10, 40, 36, 46, 21 ]
colors = [ "red", "yellow", "green",
           "blue", "purple" ]
plt.bar(labels, yValues, color=colors)
plt.show()
```



Adding xlabel and ylabel

```
labels = [ "A", "B", "C", "D", "E" ]
yValues = [ 10, 40, 36, 46, 21 ]
colors = [ "red", "yellow", "green",
           "blue", "purple" ]
plt.bar(labels, yValues, color=colors)

plt.xlabel("Product Categories")
plt.ylabel("# Purchased")

plt.show()
```



Don't Memorize – Use the Website!

There are a *ton* of visualizations you can draw in Matplotlib, and hundreds of ways to customize them. It isn't productive to try to memorize all of them.

Instead, use the documentation! Matplotlib's website is very well organized and has tons of great examples: https://matplotlib.org/

When you want to create a visualization, start by searching the **API** and the **pre-built examples** to find which methods might do what you need.

Any Question(s)?